```
void graph :: toposort () {
    Stack <int > Stack ;
    bool * visited = new bool [v];

    for (int i=0; i<v; i++)
            visited [i] = false ;

    for (int i=0; i<v; i++)
            if (visited[i] == false)
            topoutil (i, visited , stack);
    while (stack.empty () == false) {
            cout << stack. top();
            Stack . pop();
    }
}

int main () {
    int n, v1, v2;
    cin >> n ;
    graph g(n)
```

5

SET Operations

① Intersection ② Union ③ Equality ④ Set Differences

① Struct node * get intersection (node* head1, node* head2) {

```
    unordered_map <int, bool >m ;
    Struct node *result = NULL;
    node *p = head1 ;
    while (p! = NULL) {
            m[p→data] = true;
            p = p→ next;
    }
    p = head2;
    while ( p! = NULL) {
            if (m[p→ data] == true) {
                    push (& result, p→data)
                    }
            p = p→ next;
    }
    return result;
}
```

② Struct node * get_union (node * head1 , node * head 2){

```
    Set <int > union_list ;
    node *p= head 1 ;
    while ( p! = NULL) {
            union_list . insert (p→ date) ;
            p = p→ next ;
    }
    p = head2;
    while (p! = NULL) {
            union_list . insert (p→ data) ;
            p=p→ next;
    }
```

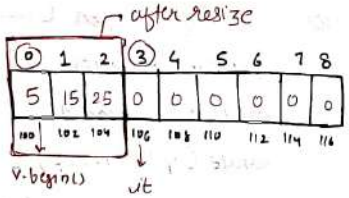
Scanned with OKEN Scanner

```
node * result = NULL;
for ( auto i : union_list ) {
        push (&result, i);
}
return result;
}
```

③ Set equality

```
int main() {
    set <int> m1, m2, m3;
    int i;
    for (i=0; i<3; i++) {
        m1.insert(i);
        m2.insert(i*i);
        m3.insert(i);
    }
    if(m1 == m3)
        cout << " Set m1 and M3 are equal ";
    else
        cout << " Not equal ";
}
```

④ Set difference




```
int main() {
    int first [ ] = {5,10,15,20,25};
    int second [ ] = { 50, 40, 30, 20, 10};
    vector <int> v(10);
    vector <int>:: iterator it;
    sort (first, first+5);
    sort (second, second+5);
    it = set_difference (first, first+5, second, second+5, v.begin());
    v.resize (it - v.begin());
    cout << " The diff "<< (v.size()) << " elements : \n" <<
    for (it = v.begin(); it != v.end(); it++)
```

S1 - S2 = 0 - 3 → -3

→ Representation of sets

→ There are 4 major ways to represent sets.

(i) Linked List [single LL]

(ii) Trees [BST, AVL, Red-black, Splay and B-tree]

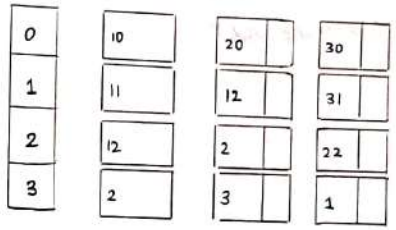
(iii) Hashing

(iv) Bit-vectors

Hashing representation : sets can be efficiently represented using hashing in C++.

→ The SPL provides the unordered set class which is a set implementation based on hashing.

S1 : h(K) = K%10



Scanned with OKEN Scanner

Key advantages :

→ That it provides near constant time avg. complexity for insertion deletion and searching operation.

→ It uses a hash table to internally store the elements making it highly efficient for managing sets.

Eg :

```
#include<unordered_set>
int main() {
  unordered_set<int> myset;
  myset.insert(5);
  myset.insert(3);
  myset.insert(7);
  int valuetofind = 3;
  if (myset.find(valuetofind)! = myset.end()) {
        cout << valuetofind << "is in the set";
        }
  else {
        cout << "Not present";
        }
}
```

Bit Vectors / Bit Set / Bit Array

→ It's a DS that represents a collection of binary values compacted, each element in a bit-vector corresponds to a single bit and these bits collectively represent a sequence of binary values.

→ The visited array in graph is an example of bit vector or bit map.

→ This implementation is efficient in terms of memory usage and provides constant time complexity for insertion, removal and membership checking operation, however it is limited by the range of elements that can be represented in the bit vector.

Q) Compute the sets of no. b/w 0 – 15 that are both prime and odd.

Bit set	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
prime	0	0	1	1	0	1	0	1	0	0	0	1	0	1	0	0
odd	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
prime & odd	0	0	0	1	0	1	0	1	0	0	0	1	0	1	0	0

3 5 7 11 13

$$\left\{ \begin{array}{ll} S1 \cup S2 = S1 \,||\, S2 & S1 \text{ AND } S2 = S1 \,\&\&\, S2 \\ S1 \cap S2 = S1 \,\&\&\, S2 & S1 - S2 = S1 \cap \sim S2 \end{array} \right\}$$

Scanned with OKEN Scanner

Q) BFS is started on a BT beginning from root vertex there is a vertex (t) at a distance (4) from root. If (t) is the (nth) vertex in BFS traversal then find the max^m and min^m possible value of (n). (4, 15)

Q) Consider the hash table with (9 slots). The hash f^n ($h(k) = k \% 9$) the collision are resolved by chaining, the following (9 key) are inserted in order:

[5, 28, 19, 15, 20, 33, 12, 17, 10]

Then find the max^m, min^m & avg. chain length in the hash table.

(a) (3, 0, 1) (b) (4, 0, 1)

(b) (3, 3, 3) (d) (3, 0, 2)

Q) Kanak has of a manager his job is to look at overal pic, he has already noted that (82% of orders) are for (17% of products). He would like to periodically peek at a list ordered by product no. of recently purchased item give his choice for a DS which will lead to be most efficient tree to handle the task and explain his reason for the chosen DS also write the algo for his choice.

Q) How many simple distint graphs can be possible with upto 4 nodes.

for no. of node	1	2	3	4
no. of edges :	0	1	3	6

no. of graphs : 2^0 2^1 2^3 2^6

$1 + 2 + 8 + 64$

$\rightarrow (75)$

Q) Design a hash f^n for the scheme so that it does not suffer from both primary & sec. clustering in add^n, the no. of probing sequences derived from it has to be no less than O (m3)

Scanned with OKEN Scanner

ADSA Pre-Uni Sample paper :

Q1) (i) use bit vector to computes sets of both prime & odd b/w 0-15

bit vector :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
prime bit :	0	0	1	1	0	1	0	1	0	0	0	1	0	1	0	0
odd bit :	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Prime & odd prime && odd :	0	0	0	$\boxed{1}$	0	$\boxed{1}$	0	$\boxed{1}$	0	0	0	$\boxed{1}$	0	$\boxed{1}$	0	0

Set of both prime & odd : (3, 5, 7, 11, 13)

(ii) possible distinct simple graphs upto 4 nodes.

nodes :	1	2	3	4	
possible max^m edges :	0	1	3	6	$\rightarrow \frac{n(n-1)}{2}$

possible max^m simple graphs : $1 + 2 + 8 + 64 = \boxed{75}$

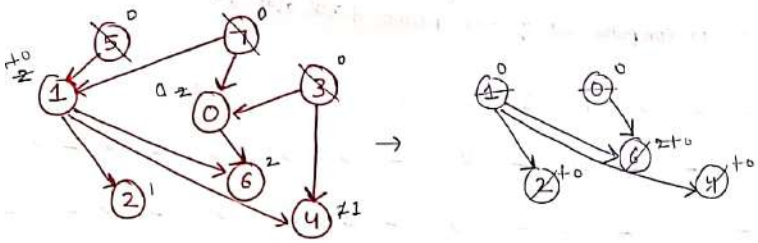
(iii) Ford Fulkerson algo

fordfulkerson (graph, source, sink)

1) Initialize flow = 0.
2) while (there exists an augmented path (P) in the residual graph)
   [2.1] Find the bottleneck capacity.
   [2.2] Augment the path P in residual graph using bottle neck capacity
   [2.3] Update the residual graph.
3) Return the maximum flow.

Scanned with OKEN Scanner

(iv) Kahn's algo for topological ordering :

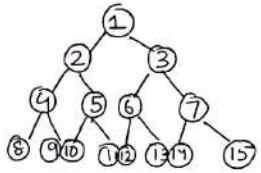


5	7	3

5	7	3	0	1	2	6	4

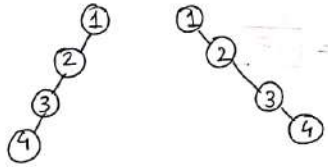
(v) distance : 4 level wise

max: max distance is when there is a complete binary tree



→ $n = 15$ max

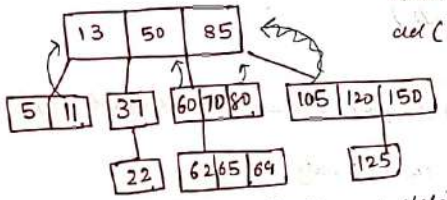
min: min n is when it's a skewed ~~gra~~ tree



→ $n_{min} = 4$

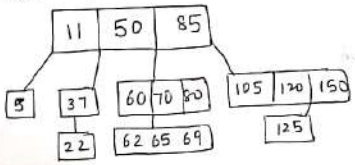
order 4: max child = 4 min child = 2
max key = 3 , min key = 1

(vi)

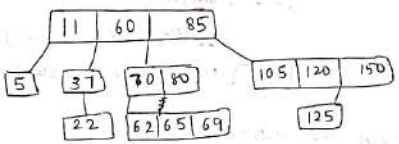


del (13,50,85)

delete (13): replace inorder predecessor



delete (50): replace using inorder successor



delete (85): replace using inorder ~~successor~~ predecessor

(vii) 10 buckets , linear probing , $H(k) = k \cdot 1 \cdot 10$

keys = [43, 165, 62, 123, 142] what is loc. of 142

linear probing: $H(k,i) = [H(k) + i] \% m$ → size

$H(k,i)$ ↓ probe no. ; $H(k)$ ↓ hash fn ; i ↓ index (0-n)

$H(43,0) = (43 \cdot 1 \cdot 10 + 0) \cdot 1 \cdot 10 \to 3$

$H(165,0) = (165 \cdot 1 \cdot 10 + 0) \cdot 110 \to 5$

$H(62,0) = (62 \cdot 1 \cdot 0 + 0) \cdot 1 \cdot 10 \to 2$

$H(123,0) = (123 \cdot 1 \cdot 10 + 0) \cdot 1 \cdot 10 \to 3$ (collission)

↳ $H(123,1) = 4$

$H(142,0) = (142 \cdot 1 \cdot 0 + 0) \cdot 1 \cdot 0 \to 2$ (collission)

↳ $H(142,1) = 3$ (collission)

↳ $H(142,2) = 4$

↳ $H(142,3) = 5$

↳ $H(142,4) = 6$

→ 142 is at 6th index | 7th posn

Key	Location	Probe
43	3	1
165	5	1
62	2	1
123	[3] 4	2
142	[3 4 5] 6	5

0	
1	
2	62
3	43
4	123
5	165
6	142
7	
8	
9	

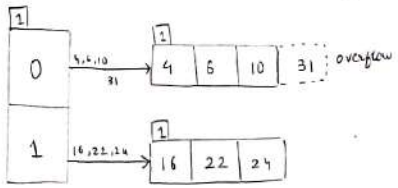
Q2) a) hash fn : X LSB's ∧ X MSB's (Extendible hashing) BS = 3

16 : 10000 22 : 10110 31 : 11111 20 : 10100

4 : 00100 24 : 11000 7 : 00111 26 : 11010

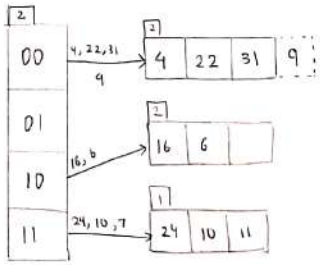
6 : 00110 10 : 01010 9 : 01001

Pass 1 :



16 : 1 ^ 0 = 1 10 : 0
4 : 0 31 : 0
6 : 0
22 : 1
24 : 1

Pass 2 :



16 : 10 00 → 10
4 : 00 00 00
6 : 00 10 10
22 : 10 10 00
24 : 11 00 11
10 : 01 10 11
31 : 11 11 00
7 : 00 11 11
9 : 01 01 00
20 : 10 00 10
26 : 11 10 01

Scanned with OKEN Scanner

INTRODUCTION

- **The File** is a collection of records. Using the primary key, we can access the records. The type and frequency of access can be determined by the type of file organization which was used for a given set of records.
- **File organization** is a logical relationship among various records. This method defines how file records are mapped onto disk blocks.
- File organization is used to describe the way in which the records are stored in terms of blocks, and the blocks are placed on the storage medium.
- The first approach to map the database to the file is to use the several files and store only one fixed length record in any given file. An alternative approach is to structure our files so that we can contain multiple lengths for records.
- **Files of fixed length records are easier to implement than the files of variable length records.**

INTRODUCTION

Objectives of File Organization in Data Structure

- It contains an **optimal selection of records**, i.e., records can be selected as fast as possible.
- To perform insert, delete or update transaction on the records should be quick and easy.
- The duplicate records cannot be induced as a result of insert, update or delete.
- For the minimal cost of storage, records should be stored efficiently.

INTRODUCTION

- **A file** is a collection of records which holds information of similar data. Almost all information stored in a computer system is a file.

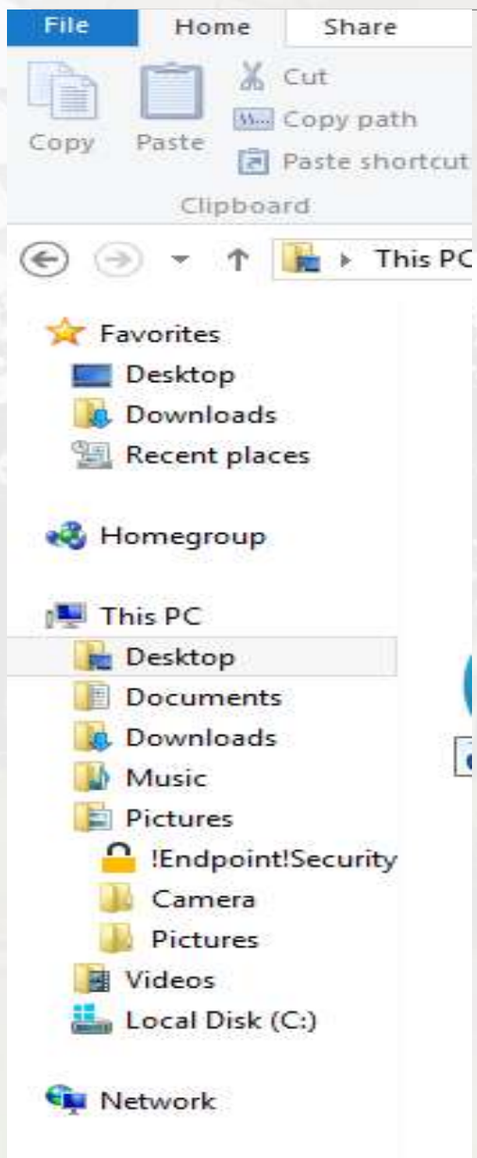
- **Need of file organization :**

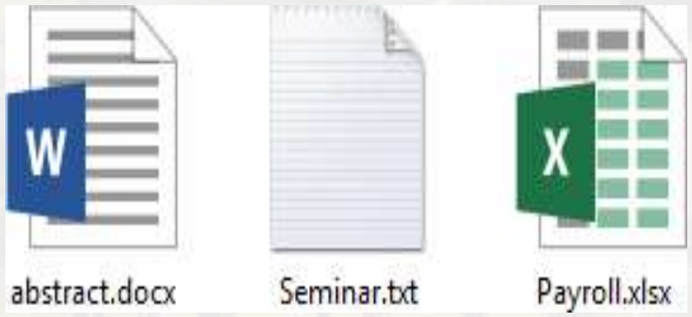
Way to organize & store data in a manner which determines standard method of data access, efficiency and flexibility.

INTRODUCTION

- **Different types o file available :**

e.g. text files, program files, directory files etc





Directory files

INTRODUCTION

Types of files supported by 'C'

- A stream refers to the flow of data from one place (storage) to another (program) or vice-versa.
- C can handle files as Stream-oriented data and System oriented data.

- **There are two types of streams Text & Binary streams**
- **Text Stream**

1. It consists of sequence of characters and each line is terminated by a newline character.
2. The I/O operations like buffering, data conversions, etc. take place automatically.

INTRODUCTION

Types of files supported by 'C'

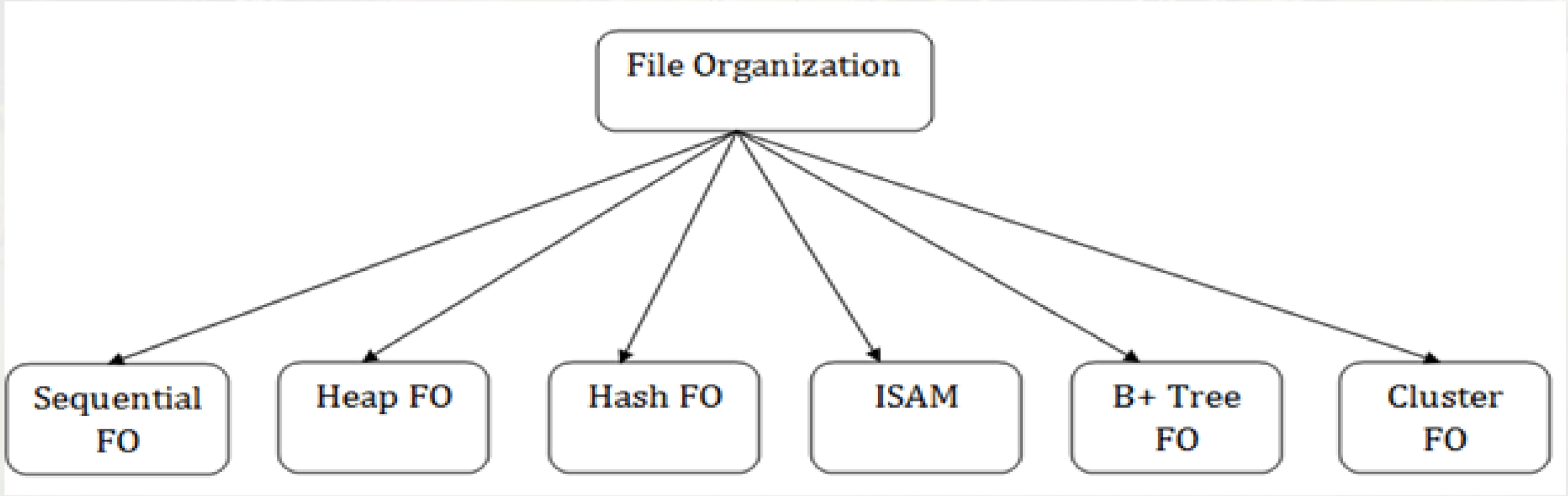
There are two types of streams Text & Binary streams

- **Binary Stream**

1. It is a series of bytes.
2. Binary streams are primarily used for non-textual data, which is required to keep exact contents of the file.
3. These are System-oriented data files that are more closely associated with the OS and data stored in memory without converting into text format.

Types of File Organization

- **Types of file organization:**
- File organization contains various methods. These particular methods have pros and cons on the basis of access or selection. In the file organization, the programmer decides the best-suited file organization method according to his requirement.
- **Types of file organization are as follows:**



Types of File Organization

- Sequential file organization
- Indexed File
- Direct Access File
- Heap file organization
- Hash file organization
- B+ file organization
- Indexed sequential access method (ISAM)
- Cluster file organization

Types of File Organization

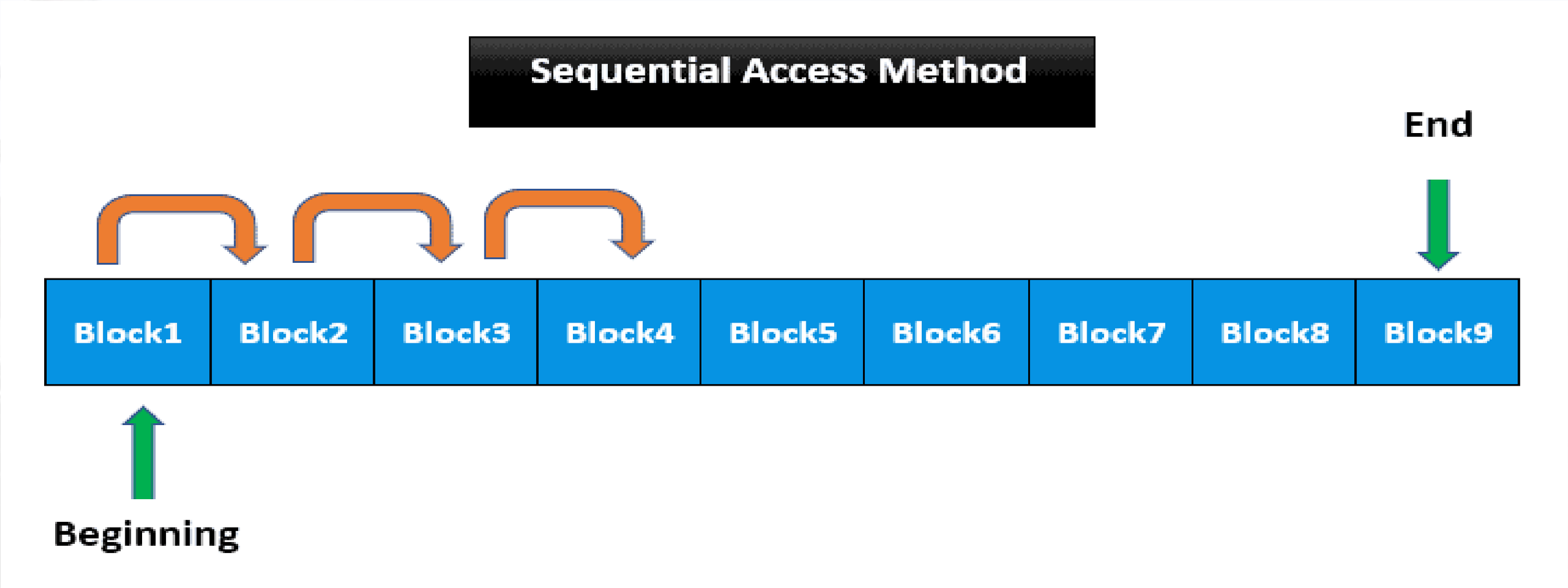
- **Sequential file organization :**

Sequential file access is the most straightforward method of accessing files. This method accesses data as one record at a time by starting from the beginning of the file to its end. Moreover, the records are read or written in the order they appear in the file.

Sequential file access is best suited for applications that linearly process data, such as reading or writing data to a log file or processing data in batch operations. For example, when analyzing a large dataset, it may be beneficial to write the data to a sequential file and then read it in one record at a time to perform the desired analysis.

Types of File Organization

- **Sequential file organization :**



The main advantage of sequential file access is its simplicity, which makes it easy to implement and use. In contrast, its main disadvantage is that it can be slow and inefficient for random access operations or when working with large files.

Types of File Organization

Sequential file organization :

- **Advantages of sequential file**

1. • It is simple to program and easy to design.
2. • Sequential file is best use if storage space.

- **Disadvantages of sequential file**

1. • Sequential file is time consuming process.
2. • It has high data redundancy.
3. • Random searching is not possible.

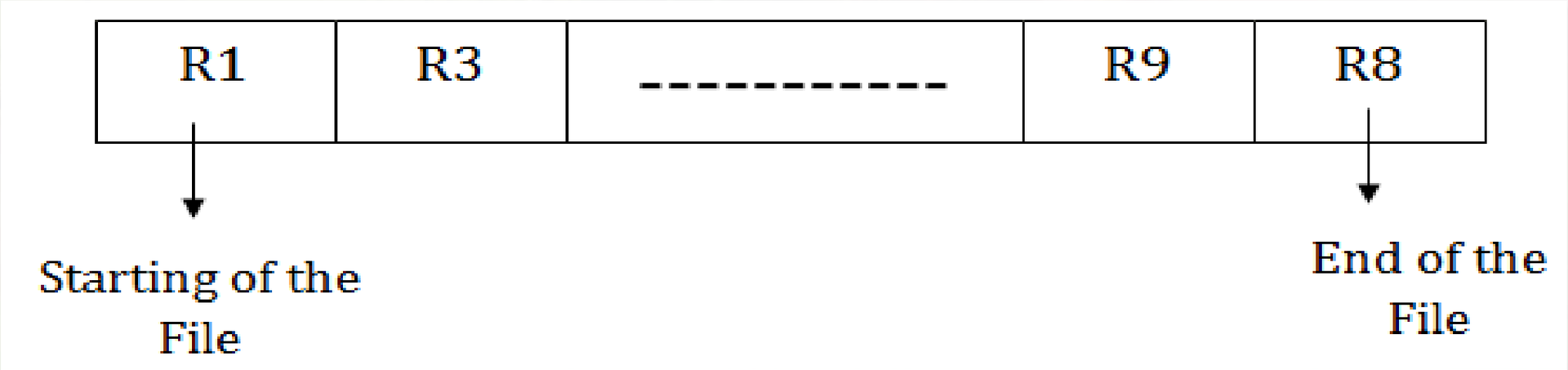
Types of File Organization

- **Sequential file organization :**

1. Pile File Method

It is a quite simple method. In this method, we store the record in a sequence, i.e., one after another. Here, the record will be inserted in the order in which they are inserted into tables.

In case of updating or deleting of any record, the record will be searched in the memory blocks. When it is found, then it will be marked for deleting, and the new record is inserted.



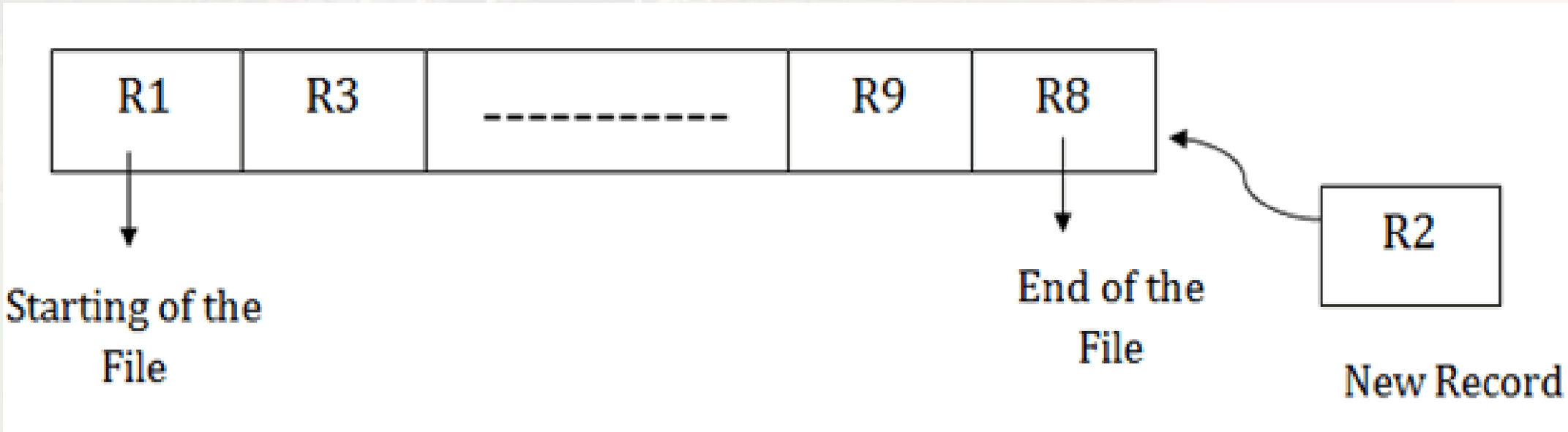
Types of File Organization

- **Sequential file organization :**

1. Pile File Method

Insertion of the new record:

Suppose we have four records R1, R3 and so on upto R9 and R8 in a sequence. Hence, records are nothing but a row in the table. Suppose we want to insert a new record R2 in the sequence, then it will be placed at the end of the file. Here, records are nothing but a row in any table.



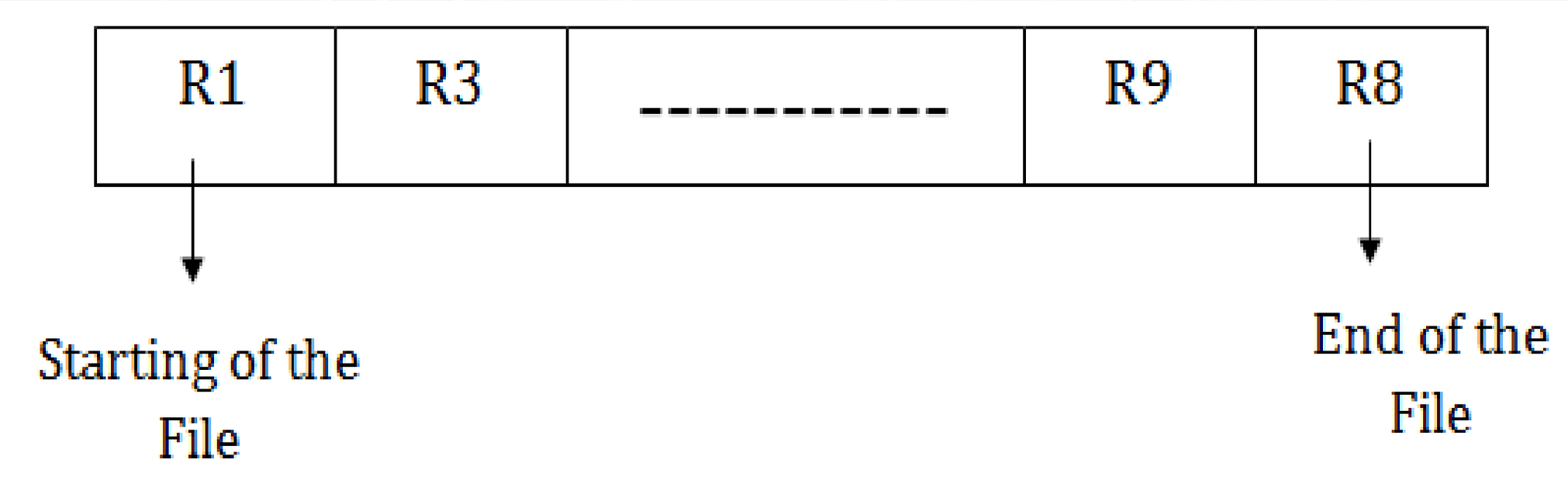
Types of File Organization

- **Sequential file organization :**

2. Sorted File Method:

In this method, the new record is always inserted at the file's end, and then it will sort the sequence in ascending or descending order. Sorting of records is based on any primary key or any other key.

In the case of modification of any record, it will update the record and then sort the file, and lastly, the updated record is placed in the right place.



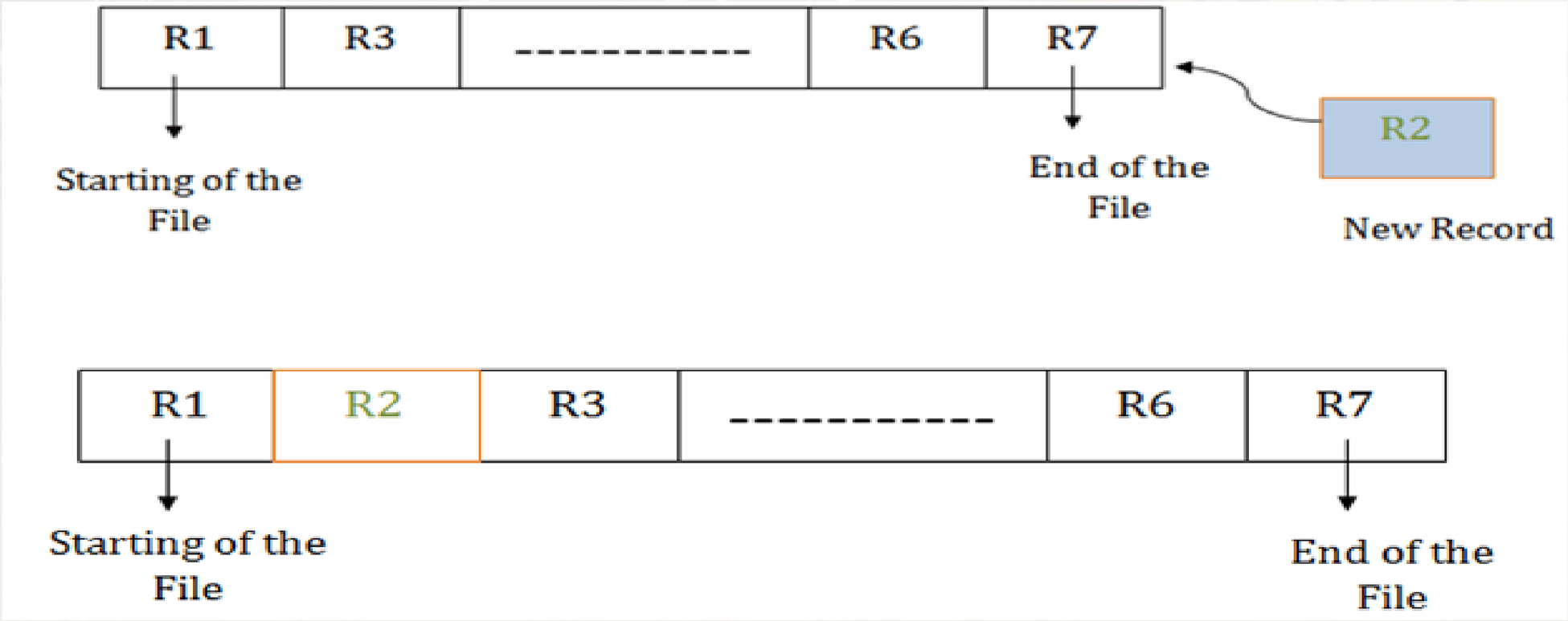
Types of File Organization

- **Sequential file organization :**

2. Sorted File Method:

Insertion of the new record:

Suppose there is a preexisting sorted sequence of four records R1, R3 and so on upto R6 and R7. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file, and then it will sort the sequence..



Types of File Organization

- **Sequential file organization :**
- **Advantages of Sequential File Organization**

1. Fast and efficient method for huge amounts of data.
2. In this method, files can be easily stored in cheaper storage mechanism like magnetic tapes.
3. It is simple in design. It requires no much effort to store the data.
4. Files can be easily stored in magnetic tapes i.e. cheaper storage mechanism.

- **Disadvantages of Sequential File Organization**

1. Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
2. The sorted file method is inefficient as it takes time and space for sorting records.

Types of File Organization

- **Indexed File Organization :**
- Indexed sequential access file combines both sequential file and direct access file organization.
- In indexed sequential access file, records are stored randomly on a direct access device such as magnetic disk by a primary key.
- This file have multiple keys. These keys can be alphanumeric in which the records are ordered is called primary key.
- The data can be access either sequentially or randomly using the index. The index is stored in a file and read into memory when the file is opened

Types of File Organization

- **Indexed File Organization :**

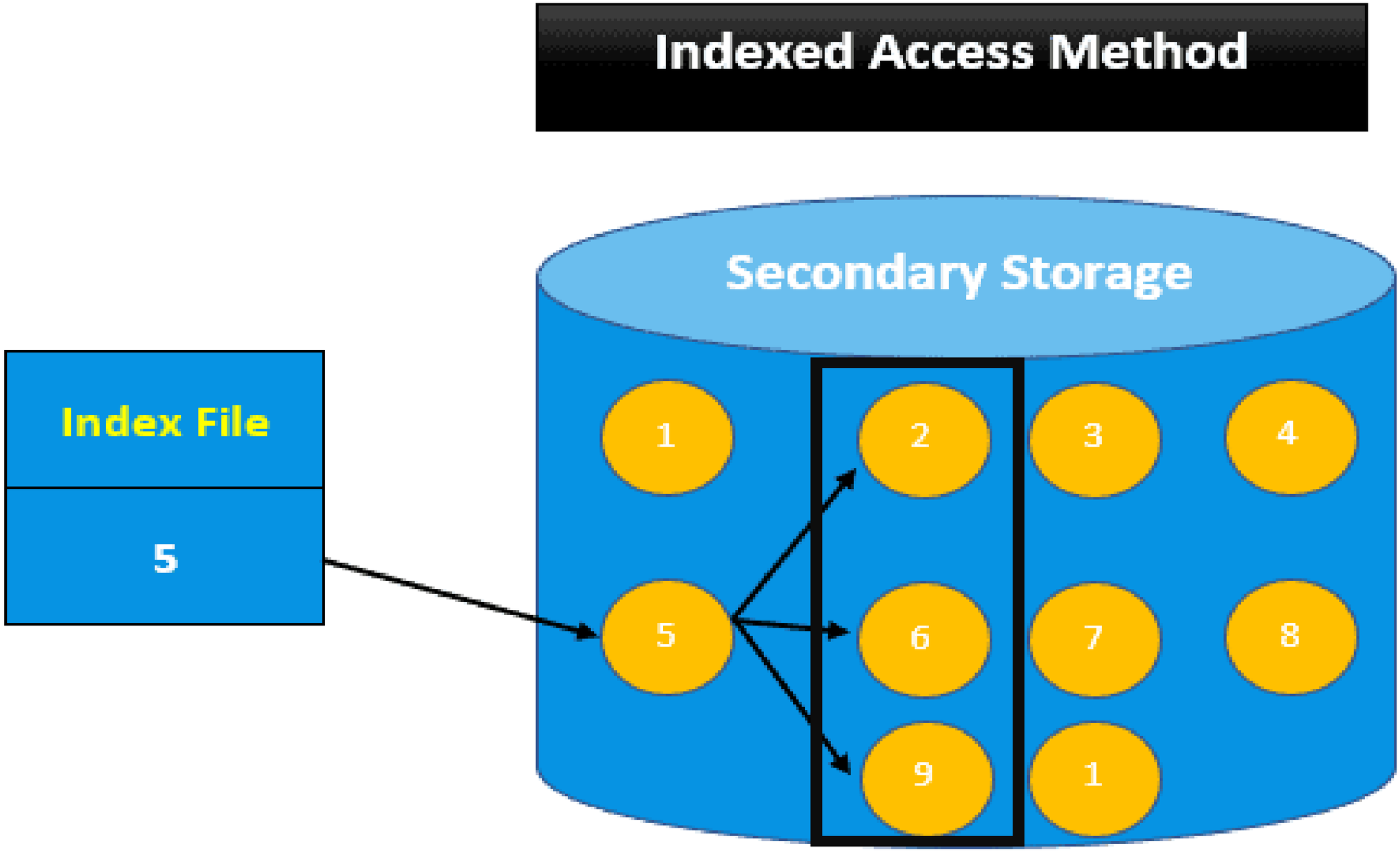
Indexed file access is a method that incorporates the benefits of both sequential and direct file access. This method involves creating an index file that maps logical keys or data elements to their corresponding physical addresses within the file. Moreover, the system stores the index separately from the data file, enabling quick access to locate the desired data.

Indexed file access is best suited for applications that require fast access to particular data elements within a large file. For example, in a file system, we may need to access specific files based on their name or location. The index created for the file system allows quick access to the file's physical location, enabling efficient access.

The **main advantage of indexed file** access is its speed and efficiency for random and sequential access operations. But**, its main disadvantage** is that it requires additional storage space for the index, which can increase the cost and complexity of the system.

Types of File Organization

- **Indexed File Organization :**

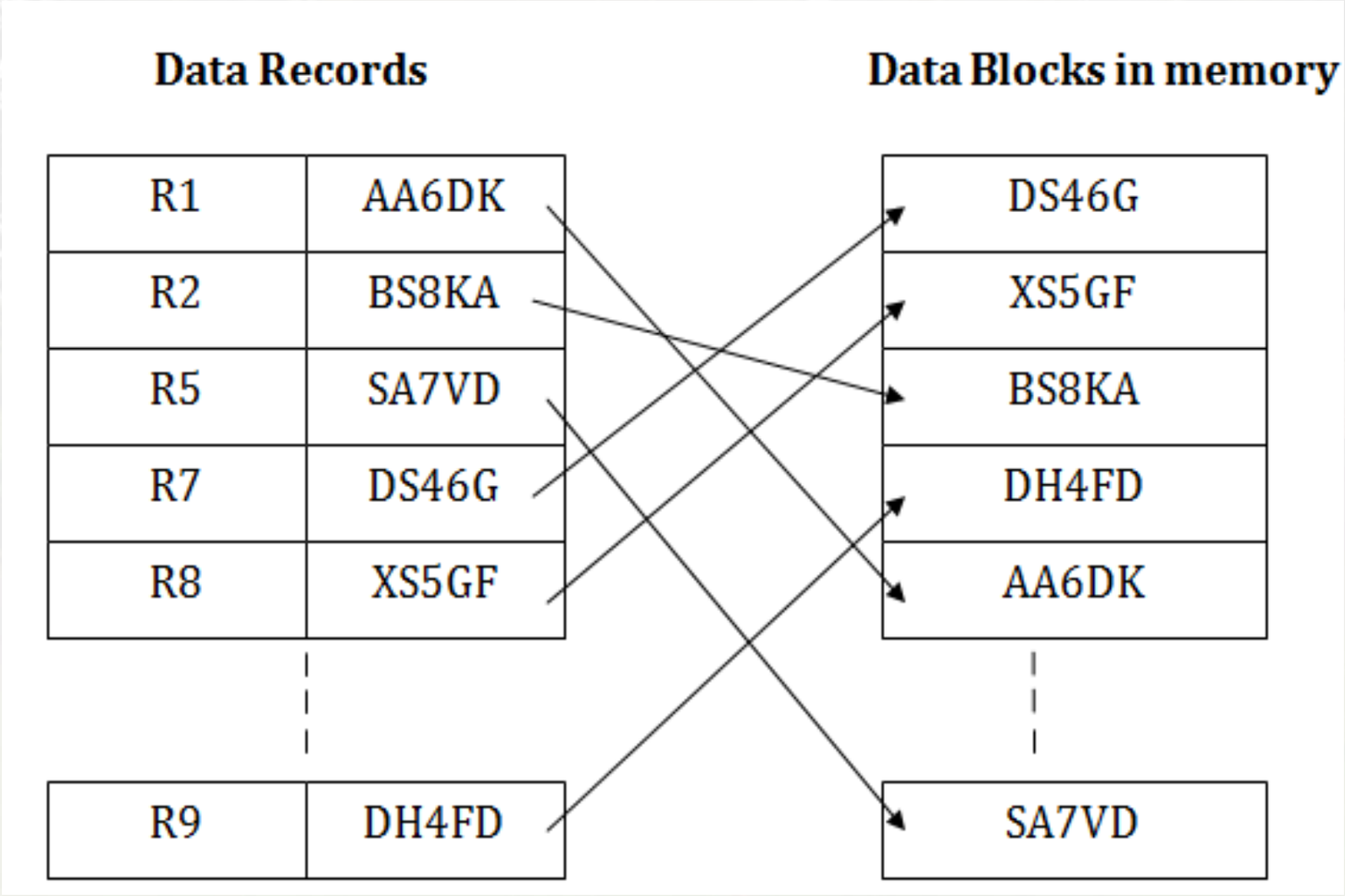


Types of File Organization

- **Indexed Sequential Access Method (ISAM) file organization :**

ISAM method is an advanced sequential file organization. In this method, records are stored in the file using the primary key. An index value is generated for each primary key and mapped with the record. This index contains the address of the record in the file.

If any record has to be retrieved based on its index value, then the address of the data block is fetched and the record is retrieved from the memory.



Types of File Organization

- **Indexed Sequential Access Method (ISAM) file organization :**
- **Advantages :**

• In indexed sequential access file, sequential file and random file access is possible.

• It accesses the records very fast if the index table is properly organized.

• The records can be inserted in the middle of the file.

• It provides quick access for sequential and direct processing.

• It reduces the degree of the sequential search.

Types of File Organization

- **Indexed Sequential Access Method (ISAM) file organization :**
- **Disadvantages :**
- Indexed sequential access file requires unique keys and periodic reorganization.
- Indexed sequential access file takes longer time to search the index for the data access or retrieval.
- It requires more storage space.
- It is expensive because it requires special software.
- It is less efficient in the use of storage space as compared to other file organizations.

Indexing in File Organization

What is Indexing?

Indexing is a data structure technique which allows you to quickly retrieve records from a database file. An Index is a small table having only two columns. The first column comprises a copy of the primary or candidate key of a table. Its second column contains a set of pointers for holding the address of the disk block where that specific key value stored.

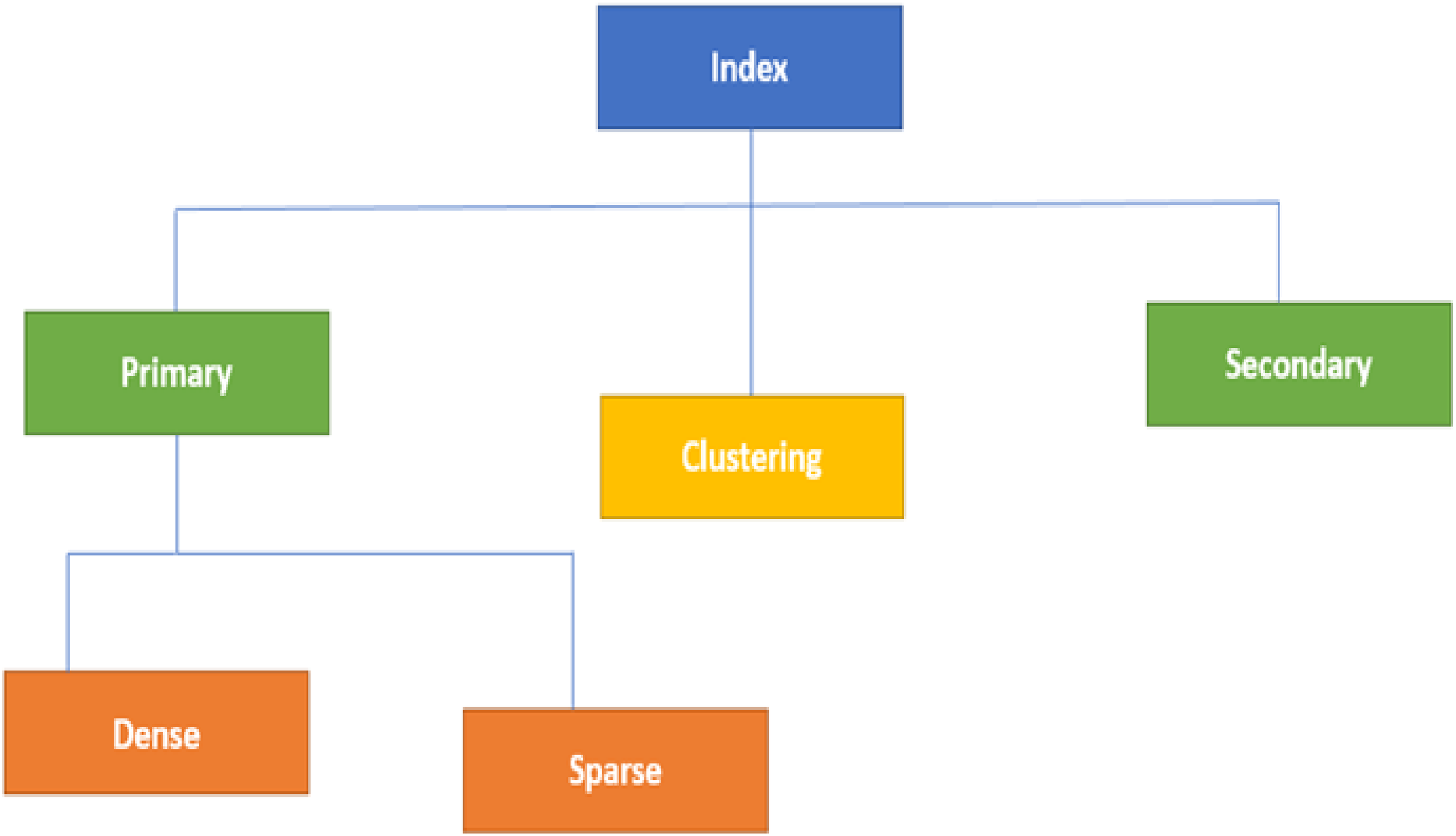
An index –

Takes a search key as input

Efficiently returns a collection of matching records

Indexing in File Organization

TYPES OF INDEXING



Indexing in File Organization

Indexing in Database is defined based on its indexing attributes. Two main types of indexing methods are:

1. Primary Indexing
2. Secondary Indexing

- **Primary Index in DBMS**

Primary Index is an ordered file which is fixed length size with two fields. The first field is the same a primary key and second, field is pointed to that specific data block. In the primary Index, there is always one to one relationship between the entries in the index table.

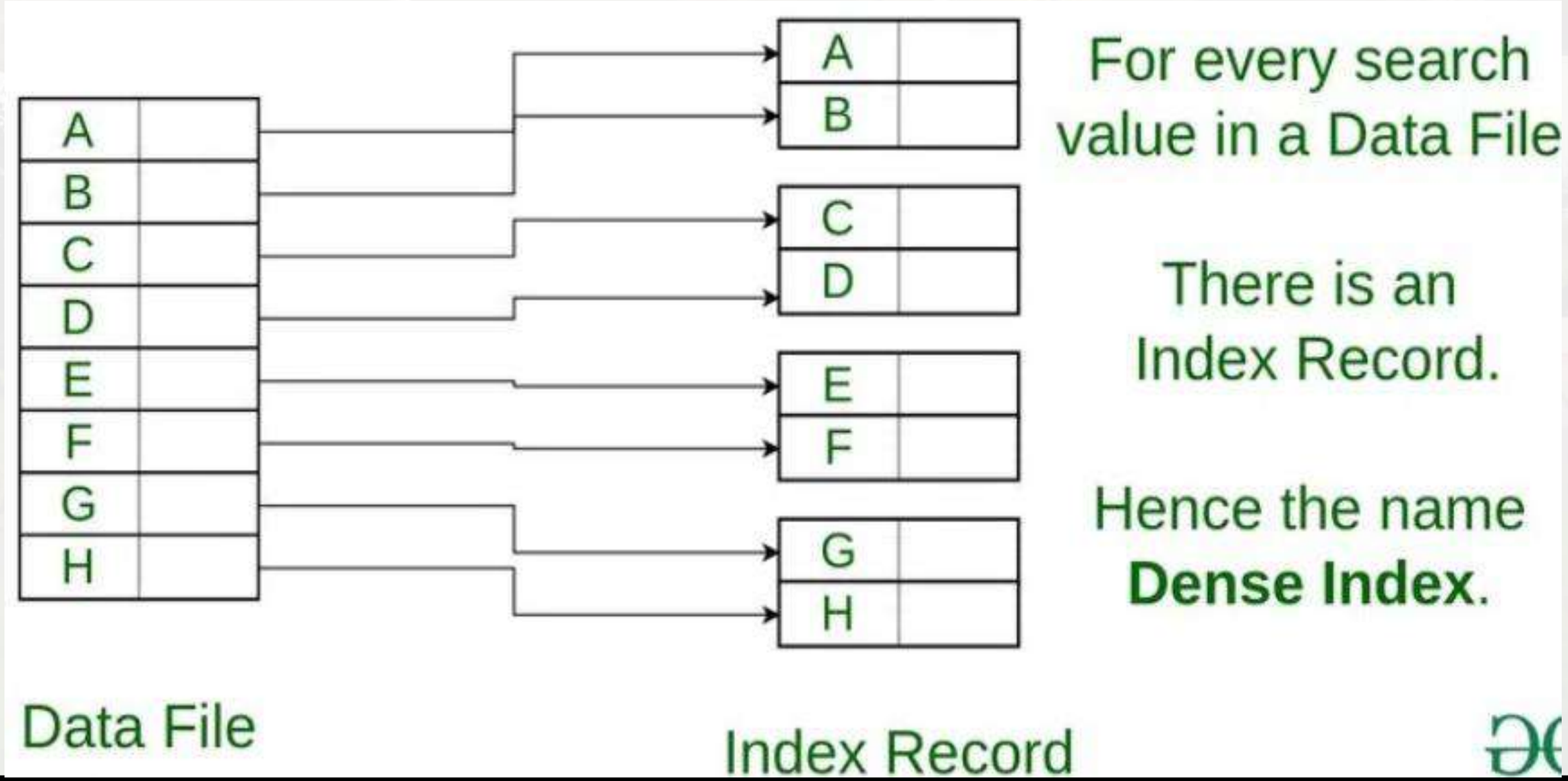
The primary Indexing in DBMS is also further divided into two types.

1. Dense Index
2. Sparse Index

Indexing in File Organization

Dense Index

In a dense index, a record is created for every search key valued in the database. This helps you to search faster but needs more space to store index records. In this Indexing, method records contain search key value and points to the real record on the disk.



Indexing in File Organization

Sparse Index :

It is an index record that appears for only some of the values in the file. Sparse Index helps you to resolve the issues of dense Indexing in DBMS.

In this method of indexing technique, a range of index columns stores the same data block address, and when data needs to be retrieved, the block address will be fetched.

However, sparse Index stores index records for only some search-key values.

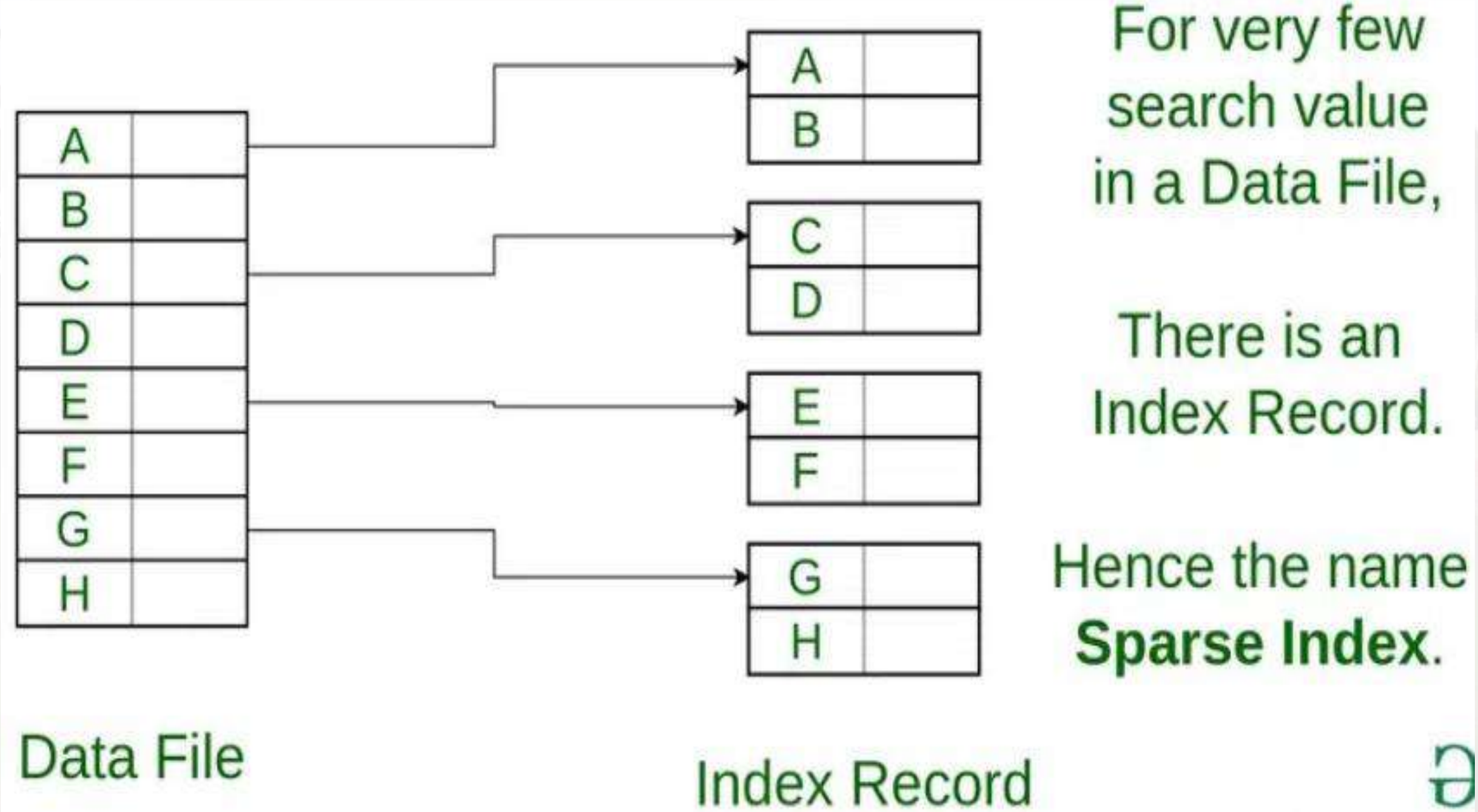
It needs less space, less maintenance overhead for insertion, and deletions but

It is slower compared to the dense Index for locating records.

Indexing in File Organization

Sparse Index :

Below is an database index Example of Sparse Index



Indexing in File Organization

Clustered Indexing: When more than two records are stored in the same file this type of storing is known as cluster indexing. By using cluster indexing we can reduce the cost of searching reason being multiple records related to the same thing are stored in one place and it also gives the frequent joining of more than two tables (records).

The clustering index is defined on an ordered data file. The data file is ordered on a non-key field. In some cases, the index is created on non-primary key columns which may not be unique for each record. In such cases, in order to identify the records faster, we will group two or more columns together to get the unique values and create an index out of them. This method is known as the clustering index. Essentially, records with similar properties are grouped together, and indexes for these groupings are formed.

Students studying each semester, for example, are grouped together. First-semester students, second-semester students, third-semester students, and so on are categorized

Indexing in File Organization

Cluster indexing :

INDEX FILE	
SEMESTER	INDEX ADDRESS
1	
2	
3	
4	
5	

Data Blocks in Memory				
100	Joseph	Alaiedon Township	20	200
101				
110	Allen	Fraser Township	20	200
111				
120	Chris	Clinton Township	21	200
121				
200	Patty	Troy	22	205
201				
210	Jack	Fraser Township	21	202
211				
300				

Indexing in File Organization

Non Clustered or Secondary Indexing :

The secondary Index in DBMS can be generated by a field which has a unique value for each record, and it should be a candidate key. It is also known as a non-clustering index.

This two-level database indexing technique is used to reduce the mapping size of the first level. For the first level, a large range of numbers is selected because of this; the mapping size always remains small.

Indexing in File Organization

Secondary Index Example

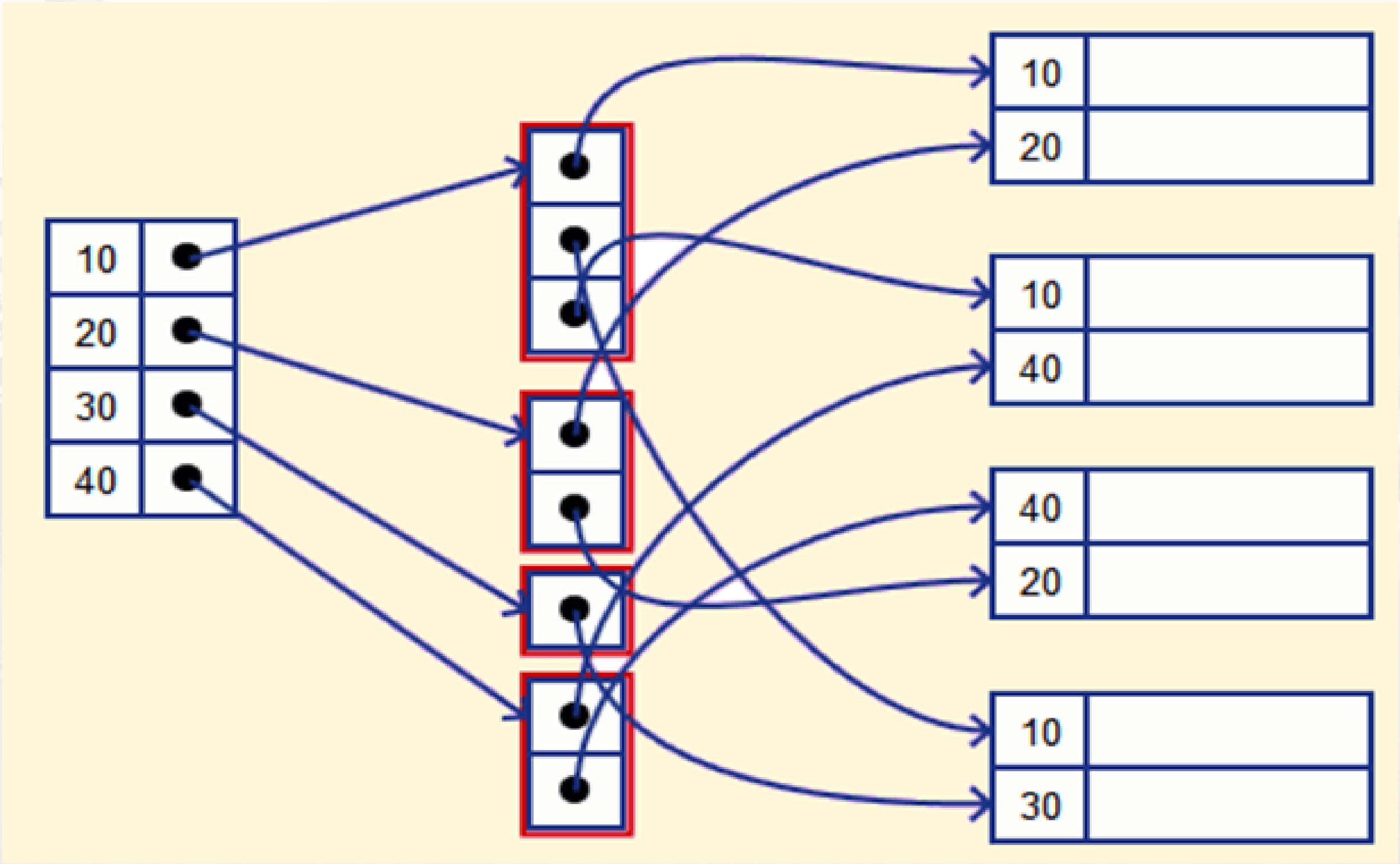
Let's understand secondary indexing with a database index example:

In a bank account database, data is stored sequentially by acc_no; you may want to find all accounts in of a specific branch of ABC bank.

Here, you can have a secondary index in DBMS for every search-key. Index record is a record point to a bucket that contains pointers to all the records with their specific search-key value.

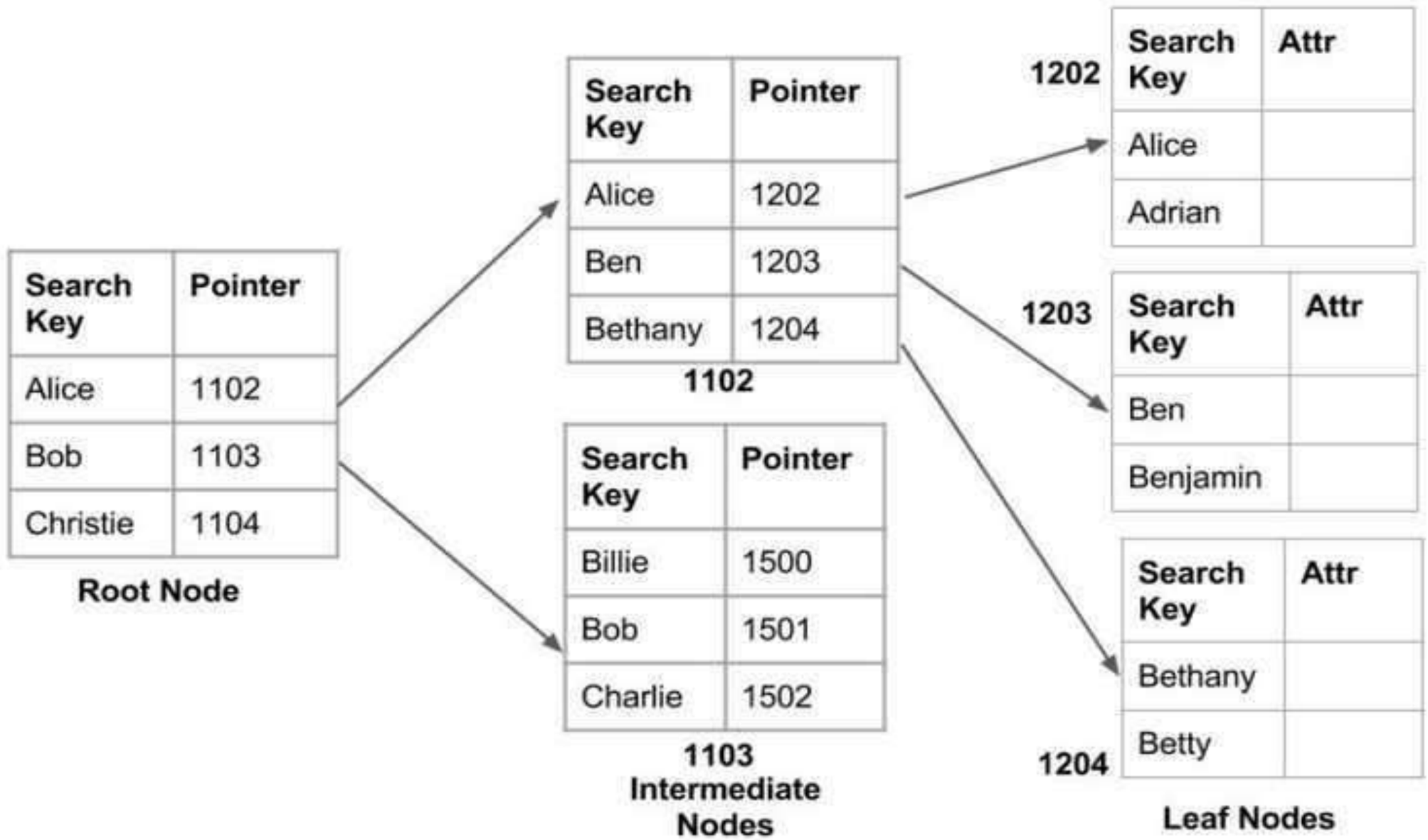
Indexing in File Organization

Secondary Index Example



Indexing in File Organization

Secondary Index Example



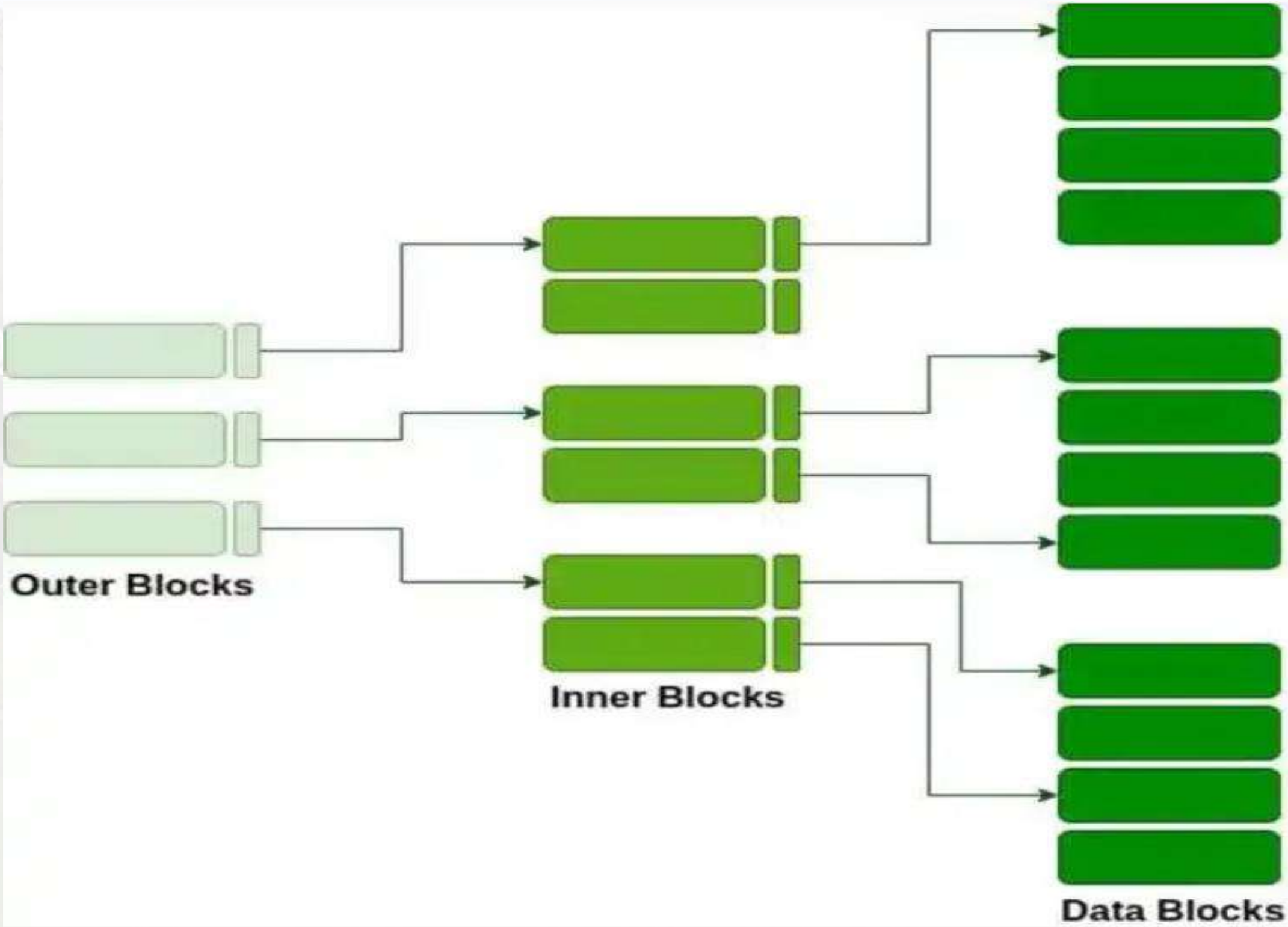
Non clustered index

Indexing in File Organization

Multilevel Indexing: With the growth of the size of the database, indices also grow. As the index is stored in the main memory, a single-level index might become too large a size to store with multiple disk accesses. The multilevel indexing segregates the main block into various smaller blocks so that the same can be stored in a single block. The outer blocks are divided into inner blocks which in turn are pointed to the data blocks. This can be easily stored in the main memory with fewer overheads.

Indexing in File Organization

Multilevel Indexing:



Indexing in File Organization

Advantages of Indexing

Important pros/ advantage of Indexing are:

1. It helps you to reduce the total number of I/O operations needed to retrieve that data, so you don't need to access a row in the database from an index structure.
2. **Offers Faster search and retrieval of data to users.**
3. Indexing also helps you to **reduce tablespace** as you don't need to link to a row in a table, as there is no need to store the ROWID in the Index. Thus you will able to reduce the tablespace.
4. You can't sort data in the lead nodes as the value of the primary key classifies it.

Indexing in File Organization

Disadvantages of Indexing :

Important drawbacks/cons of Indexing are:

1. To perform the indexing database management system, you need **a primary key** on the table with a unique value.
2. You can’t perform any other indexes in Database on the Indexed data.
3. You are not allowed to partition an index-organized table.
4. SQL Indexing Decrease performance in INSERT, DELETE, and UPDATE query.

Types of File Organization

- **Direct Access File organization :**

Direct access file is also known as random access or relative file organization.

• In direct access file, all records are stored in direct access storage device (DASD), such as hard disk. The
records are randomly placed throughout the file.

• The records does not need to be in sequence because they are updated directly and rewritten back in the
same location.

• This file organization is useful for immediate access to large amount of information. It is used in accessing
large databases.

• It is also called as hashing.

Types of File Organization

- **Direct Access File organization :**
- **Direct file access, also known as random access.**
- It allows us to access data directly from any location within the file, without the need to read or write all the records that come before it. Furthermore, this method accesses records within the file by using their physical addresses or positions.
- Direct file access is best suited for applications that require quick and efficient access to specific records or data elements within a file.
- For example, in a database application, we may need to quickly retrieve customer data based on a specific customer ID. Direct file access can quickly access the record containing the customer data without having to read through all the records that come before it.

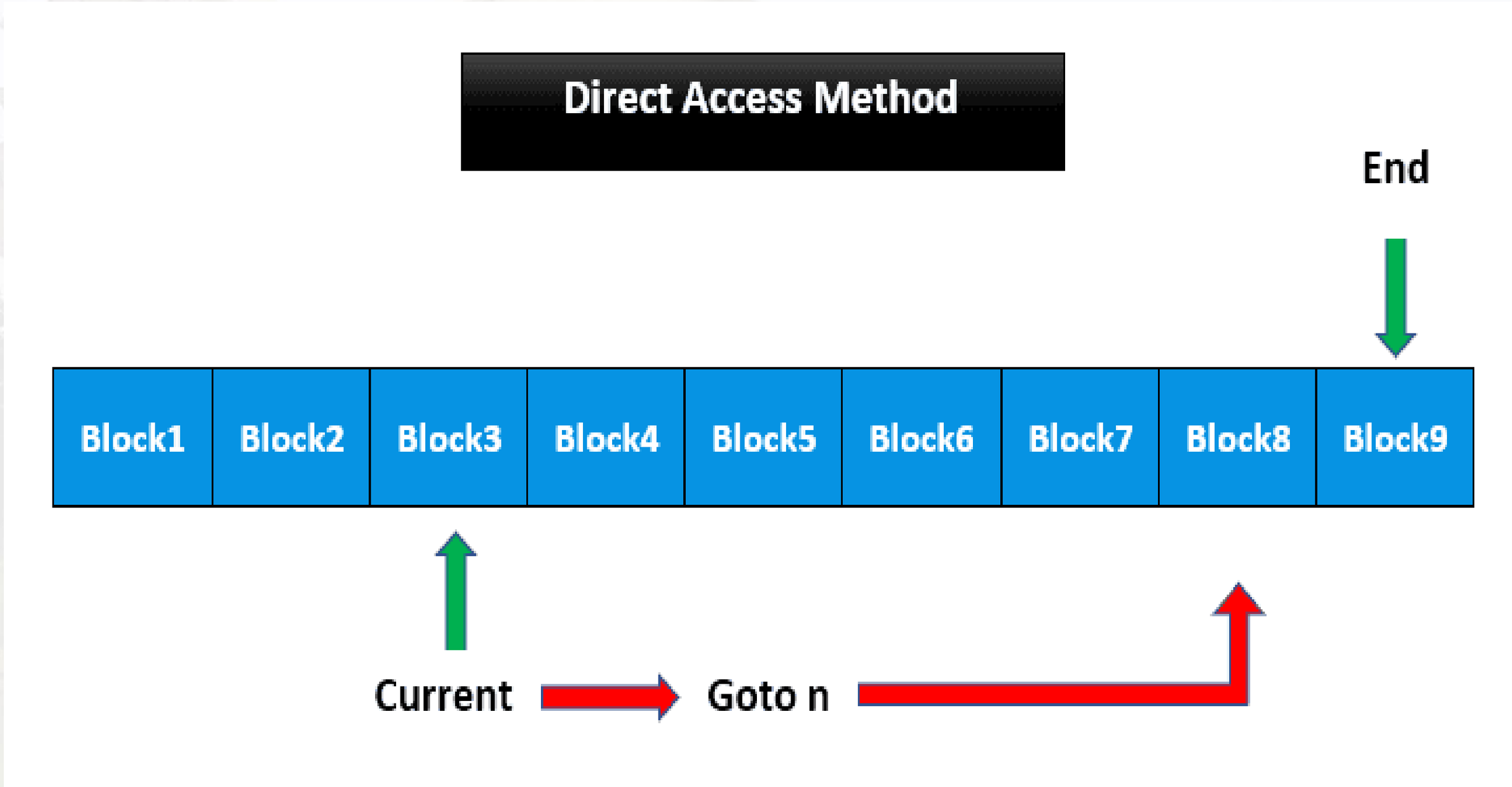
Types of File Organization

- **Direct Access File organization :**

The **main advantage** of direct file access is its speed and efficiency for random access operations. On the other hand, its **main disadvantage** is that it can be more complex and difficult to implement and use than sequential file access.

Types of File Organization

- **Direct Access File organization :**



Types of File Organization

Direct Access File Organization :

Advantages of direct access file organization

1. • Direct access file helps in online transaction processing system (OLTP) like online railway reservation system.
2. • In direct access file, sorting of the records are not required.
3. • It accesses the desired records immediately.
4. • It updates several files quickly.
5. • It has better control over record allocation.

Disadvantages of direct access file organization

1. • Direct access file does not provide back up facility.
2. • It is expensive.
3. • It has less storage space as compared to sequential file.

Comparison of file method

Method	Sequential File Access	Direct File Access	Indexed File Access
Access method	Accesses data one record at a time in sequential order from beginning to end	Accesses data directly from any location within the file, using physical addresses or positions	Uses an index that maps logical keys or data elements to their corresponding physical addresses within the file
Suited for	Applications that process data in a linear fashion, such as reading or writing data to a log file or processing data in batch operations	Applications that require quick and efficient access to specific records or data elements within a file	Applications that require fast and efficient access to specific data elements within a large file
Advantages	Simplicity and ease of implementation and use	Speed and efficiency for random access operations	Speed and efficiency for both random and sequential access operations
Disadvantages	Slow and inefficient for random access operations or when working with large files	More complex and difficult to implement and use than sequential file access	Requires additional storage space for the index, which can increase the cost and complexity of the system